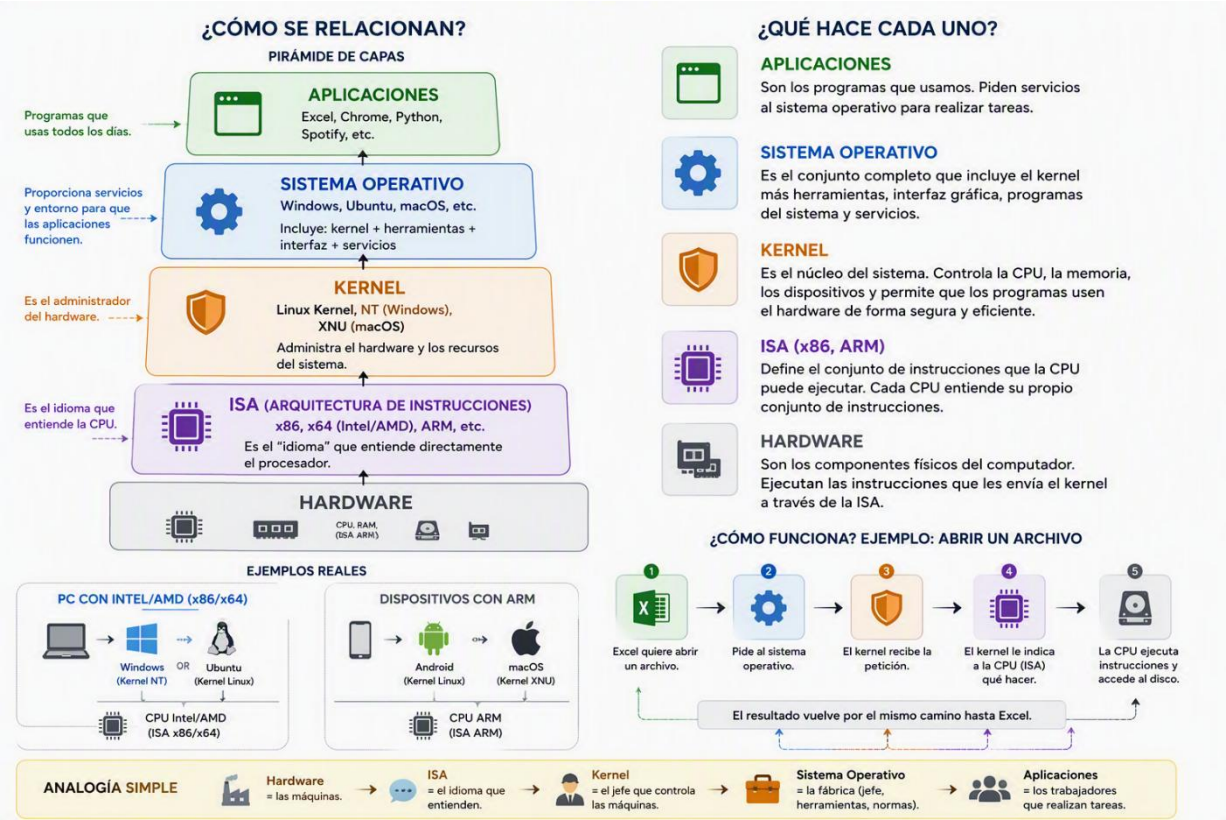
Sistemas Operativos



Las funciones esenciales de un sistema operativo pueden resumirse en:

1. Gestionar procesos.
2. Gestionar memoria.
3. Gestionar archivos.
4. Gestionar dispositivos de entrada/salida.
5. Administrar el almacenamiento.
6. Proporcionar interfaz al usuario.
7. Garantizar la seguridad.
8. Administrar recursos del sistema.
9. Gestionar las comunicaciones en red.
10. Detectar y controlar errores.

UNIX

En 1969, Ken Thompson y Dennis Ritchie crearon UNIX en Bell Labs. UNIX introdujo una visión revolucionaria de cómo debía funcionar el software:

1. Estar escrito en Lenguaje C: A diferencia de sus predecesores creados en ensamblador para una computadora específica, C permitió portar UNIX a distintos tipos de procesadores.
2. Filosofía modular: Crear programas pequeños que hagan una sola cosa bien y se conecten entre sí mediante tuberías (*pipes*).
3. El sistema de archivos unificado: "Todo es un archivo" (incluidos discos y terminales), facilitando enormemente el control del hardware.

¿Cómo derivaron de UNIX los sistemas modernos?

1. Linux (inspirado en UNIX)

En 1991, Linus Torvalds desarrolló Linux con el objetivo de disponer de un sistema operativo tipo UNIX para computadores personales basados en procesadores Intel. Debido a las restricciones de licencia del UNIX comercial, creó un nuevo kernel desde cero inspirado en los principios de diseño de UNIX.

- Diseñado para ser compatible con estándares POSIX y herramientas tradicionales de UNIX.
- Impacto: actualmente domina la gran mayoría de servidores de Internet, prácticamente todas las supercomputadoras y numerosos sistemas embebidos.

2. Android (basado en Linux)

Android surgió originalmente en la empresa Android Inc., fundada en 2003 y adquirida por Google en 2005. En 2008 se lanzó la primera versión comercial del sistema operativo. Aunque Android posee una interfaz y una arquitectura de aplicaciones propias, utiliza como base un kernel Linux adaptado para dispositivos móviles.

- Hereda del kernel Linux mecanismos de multitarea, gestión de memoria, seguridad y administración de hardware.
- Google modificó Linux para incorporar soporte para pantallas táctiles, sensores, comunicaciones móviles y gestión eficiente de energía.
- Impacto: actualmente es el sistema operativo móvil más utilizado del mundo.

3. macOS e iOS (descendientes de la familia UNIX)

Apple desarrolló sus sistemas modernos a partir de NeXTSTEP, el sistema operativo de la empresa NeXT fundada por Steve Jobs. NeXTSTEP estaba basado en el microkernel Mach e incorporaba tecnologías procedentes de BSD.

- En 1996 Apple adquirió NeXT.
- Esta tecnología evolucionó hacia Mac OS X, conocido actualmente como macOS.
- iOS deriva de la misma base tecnológica.
- macOS mantiene compatibilidad y certificación oficial UNIX.

4. Windows (una evolución independiente)

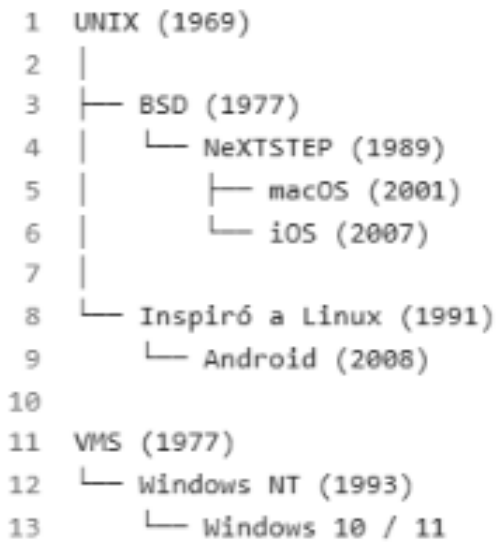
Microsoft siguió una trayectoria distinta a la de UNIX.

- Inicialmente utilizó MS-DOS, un sistema operativo con capacidades limitadas de multitarea y seguridad.
- Para crear una arquitectura moderna, contrató a Dave Cutler, quien lideró el desarrollo de Windows NT, lanzado en 1993.
- Windows 10 y Windows 11 son descendientes directos de la familia Windows NT.
- Posteriormente Microsoft incorporó WSL y WSL2 para facilitar la ejecución de aplicaciones y entornos Linux dentro de Windows.

Conclusión

Los sistemas operativos actuales proceden principalmente de tres grandes linajes tecnológicos. Linux constituye una reimplementación inspirada en UNIX; Android deriva de Linux y trasladó esa herencia al ámbito móvil; macOS e iOS descienden de tecnologías BSD y NeXTSTEP vinculadas históricamente a UNIX; y Windows NT representa una evolución independiente. De esta forma, gran parte de los dispositivos modernos, desde servidores y supercomputadoras hasta teléfonos inteligentes, utilizan sistemas que tienen una relación directa o indirecta con la tradición UNIX.

Relación histórica simplificada:



Un sistema operativo es el software que actúa como intermediario entre el usuario y el hardware de la computadora. Su función básica es permitir que las aplicaciones se ejecuten y que el usuario pueda interactuar con la máquina de manera sencilla.

Ejemplo simple: sin sistema operativo, una computadora sería solo un conjunto de circuitos y componentes sin utilidad práctica.

GNU

(Acrónimo Recursivo): Significa *GNU's Not Unix* ("GNU No es Unix"), para indicar que funcionaba como Unix pero sin usar su código privado.

El proyecto GNU aporta prácticamente todo el entorno de trabajo que rodea al motor (kernel). Se divide principalmente en cuatro grandes categorías de programas:

1. La Shell y las Utilidades Básicas (Coreutils)

Son los comandos fundamentales que usas para interactuar con la computadora a través de la terminal o la consola:

- Bash: La línea de comandos donde escribes las instrucciones.
- Comandos de archivos: Herramientas como ls (listar archivos), cp (copiar), mv (mover), rm (eliminar) o cat (leer archivos).
- Procesadores de texto: Comandos como grep, sed y awk para buscar y manipular texto.

2. Las Herramientas de Desarrollo (Para crear software)

Son los programas que permiten a los desarrolladores escribir y compilar otros programas:

- GCC (GNU Compiler Collection): El compilador que traduce el código escrito por humanos (en C, C++, etc.) a lenguaje máquina que entiende el procesador.
- GDB: El depurador (*debugger*) que ayuda a encontrar errores en el código.
- Make: Herramienta para automatizar la construcción y compilación de proyectos grandes.

3. La Biblioteca C (glibc)

Es el "puente" invisible pero vital. La GNU C Library (glibc) es un conjunto de funciones estándar en lenguaje C que permite a las aplicaciones pedirle favores al kernel (como abrir un archivo, reservar memoria o conectarse a internet). Sin esta biblioteca, las aplicaciones no sabrían cómo hablar con el kernel Linux.

4. Aplicaciones y Programas de Usuario

El proyecto también creó programas completos para el usuario final o avanzado:

- Emacs y Nano: Editores de texto.
- GIMP: Programa de edición de imágenes (originalmente creado en el entorno GNU).
- GNOME: Uno de los entornos de escritorio gráficos más populares para Linux.

GNU / LINUX

Linux

Es solamente el kernel (núcleo).

Se encarga de:

- Gestionar la memoria.
- Gestionar los procesos.
- Controlar el hardware.
- Administrar dispositivos y sistemas de archivos.

Sin programas adicionales, el kernel Linux por sí solo no forma un sistema operativo utilizable.

GNU

Aporta muchas de las herramientas necesarias para usar el sistema:

- Bash (intérprete de comandos).
- GCC (compilador).
- Coreutils (cp, ls, mv, rm, cat, etc.).
- Bibliotecas del sistema.
- Herramientas de administración.

Memoria Virtual

La **Memoria Virtual** es una técnica de gestión de memoria que permite al sistema operativo proporcionarles a los programas la ilusión de tener una memoria RAM gigantesca, continua y dedicada para cada uno, **incluso si la memoria RAM física real es mucho más pequeña o está llena.**

1. ¿Cómo funciona en la práctica?

Imagina que tu computadora tiene **8 GB de memoria RAM física**, pero quieres abrir un juego pesado o varias aplicaciones que entre todas necesitan **16 GB**.

Sin memoria virtual, el sistema simplemente colapsaría o te diría *"Memoria insuficiente"*.

Con memoria virtual, el sistema resuelve el problema dividiendo la memoria en pequeños bloques de tamaño fijo llamados **páginas** (por lo general de 4 KB) y hace lo siguiente:

1. Mantiene en la memoria RAM real solo las **páginas que los programas están usando en este preciso instante**.
2. Mueve las páginas que no se están usando activamente a un espacio reservado dentro del **disco rígido o SSD** (este espacio se llama *Archivo de paginación*, *Swap* o *Paging file*).
3. Si la CPU necesita un dato que fue enviado al disco, ocurre lo que se conoce como un **Fallo de página (*Page Fault*)**: el sistema pausa un milisegundo la ejecución, trae la página desde el disco hacia la RAM, y reanuda el trabajo.

2. ¿Quién la controla?

La memoria virtual no la controla un solo elemento, sino un **trabajo en equipo perfecto entre el Hardware y el Software**:

A. El Hardware: La MMU (*Memory Management Unit*)

- Es un circuito integrado directamente dentro de la CPU (lo vimos en el diagrama de la arquitectura x86).
- **Su trabajo:** Realiza la traducción en tiempo real. Cuando un programa pide acceder a una dirección de *"Memoria Virtual"* (la dirección lógica que ve el programador), la MMU la traduce a la dirección de *"Memoria Física"* (la posición real en los chips de la RAM).

B. El Software: El Sistema Operativo (Windows, Linux, macOS, etc.)

- Es el encargado de administrar la política de paginación.
- **Su trabajo:**
    - Construye y actualiza las **Tablas de Páginas** (*Page Tables*) que la MMU consulta.

- Decide qué páginas de datos se quedan en la RAM y cuáles se envían al almacenamiento del disco (*Swap*).
- Maneja los fallos de página (*Page Faults*) para transferir los datos del disco a la RAM cuando la MMU se lo solicita.

3. Ventajas principales para la materia:

- **Aislamiento y Protección:** Cada programa vive en su propio "mundo" de memoria virtual. Un programa malicioso o con un error no puede leer ni modificar la memoria virtual de otro programa ni del propio Sistema Operativo.
- **Más memoria disponible:** Permite ejecutar aplicaciones exigentes sin quedar limitado únicamente por los gigabytes físicos de la memoria RAM.
- **Simplificación para el programador:** El programador no tiene que preocuparse por dónde se ubicará su código en los chips de RAM físicos ni de si hay suficiente espacio contiguo; la MMU y el Sistema Operativo se encargan de acomodarlo.

Analogía MMU – MEMORIA VIRTUAL – SISTEMA OPERATIVO

Imagina una ciudad de juguetes

Hay un niño (el programa) que quiere jugar con miles de juguetes (los datos).
Pero en su habitación (la memoria RAM) **no entran todos los juguetes**.
Entonces los padres (el sistema operativo) guardan el resto en un gran depósito (el disco duro).
El niño no sabe dónde está realmente cada juguete. Solo pide:
"Quiero mi auto rojo."
Los padres son quienes buscan el juguete y se lo llevan a la habitación.
El niño piensa que **todos sus juguetes siempre están disponibles**, aunque muchos estén en el depósito.
Eso es la **memoria virtual**.

¿Qué hace la MMU?

Ahora aparece un ayudante llamado **MMU** (Memory Management Unit).
El niño dice:

"Quiero el juguete número 150."

La MMU mira un mapa y responde:

"Ese juguete realmente está en la caja 8 de la habitación."

o

"Ese juguete no está aquí, está en el depósito."

Si no está en la habitación, la MMU le avisa a los padres (el sistema operativo).

¿Qué hace el sistema operativo?

Cuando la MMU dice:

"No encontré ese juguete."

El sistema operativo hace esto:

1. Va al depósito.
2. Busca el juguete.
3. Si la habitación está llena, guarda otro juguete en el depósito.
4. Trae el juguete pedido.
5. Actualiza el mapa.
6. El niño sigue jugando sin notar nada.

Todo ocurre muy rápido.

¿Qué es la memoria virtual?

Es como darle al niño una habitación "mágica".

Aunque la habitación tenga espacio para 20 juguetes, el niño cree que puede tener 10.000.

No sabe que muchos están guardados en otro lugar.

La memoria virtual **crea la ilusión de tener mucha más memoria de la que realmente existe**.

¿Qué es una dirección virtual?

Supongamos que el niño tiene una lista:

Juguete 1
Juguete 2
Juguete 3
...
Juguete 5000

Esos números son las **direcciones virtuales**.

No indican dónde está realmente el juguete.

Solo son nombres.

La MMU los traduce a la ubicación real.

¿Qué es una dirección física?

Es la ubicación verdadera.

Por ejemplo:

Caja 3
Posición 12

o

RAM
Bloque 57

La dirección física es donde realmente está el dato.

¿Qué es la paginación?

Mover juguete por juguete sería muy lento.
Entonces todos los juguetes se guardan en **cajas**.
Cada caja tiene exactamente el mismo tamaño.
Por ejemplo:

- Caja 1
- Caja 2
- Caja 3
- Caja 4

Cada caja contiene varios juguetes.
Esas cajas se llaman **páginas**.

La RAM también está dividida en cajas iguales.
Esas cajas se llaman **marcos** (frames).
Entonces ocurre algo así:
Depósito

- Página 0
- Página 1
- Página 2
- Página 3

RAM

- Marco 0
- Marco 1
- Marco 2
- Marco 3

Una página puede colocarse en cualquier marco.

¿Cómo sabe la MMU dónde está cada página?

Tiene un mapa.
Ese mapa es la **tabla de páginas**.
Por ejemplo:

Página	**Marco**
0	3
1	0
2	No está
3	2

Si el programa pide la página 3, la MMU mira:

Página 3 → Marco 2

Entonces accede a la RAM.

Si dice:

Página 2 → No está

Se produce un **fallo de página (page fault)**.

¿Qué es un Page Fault?

Es cuando la MMU dice:

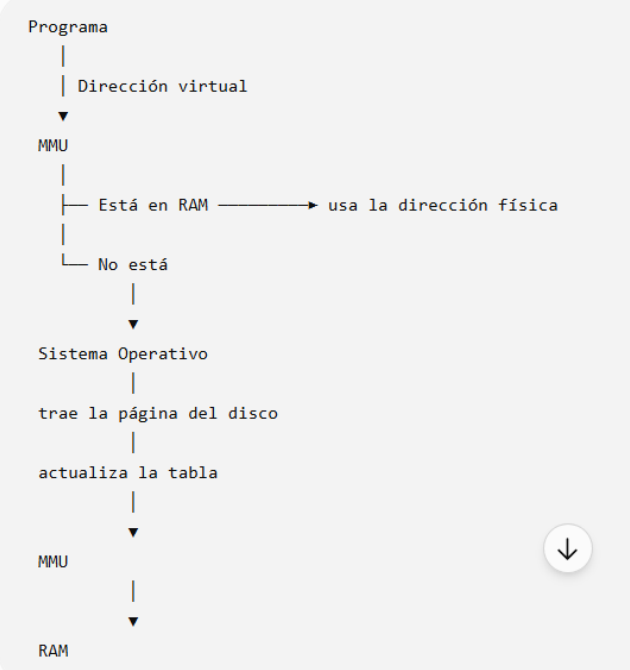
"La página que me pediste no está en la RAM."

Entonces el sistema operativo:

- pausa el programa,
- trae la página desde el disco,
- si hace falta, reemplaza otra página,
- actualiza la tabla,
- continúa el programa.

El programa casi nunca nota que ocurrió.

Resumen sencillo



Una frase para recuerdar

- **Memoria virtual:** "Hace creer que hay una memoria enorme, aunque la RAM sea pequeña."

- **MMU:** "Es el traductor que convierte las direcciones que usa el programa en direcciones reales de la RAM."
- **Sistema operativo:** "Es el encargado de traer y guardar páginas entre el disco y la RAM cuando hace falta."
- **Paginación:** "Es una forma de dividir la memoria en cajitas del mismo tamaño para moverlas fácilmente y aprovechar mejor el espacio."

Con esta analogía, se puede entender primero la idea intuitiva y luego relacionarla con los conceptos técnicos de **direcciones virtuales**, **direcciones físicas**, **MMU**, **tablas de páginas** y **fallos de página**.

Diferencia entre Memoria RAM Y Memoria Virtual

Sí, hay una diferencia fundamental. Aunque ambas se utilizan para almacenar los datos que la computadora necesita para funcionar, una es un **componente físico (hardware)** y la otra es una **técnica de gestión del sistema operativo (software)**.

Diferencias principales

Característica	Memoria RAM	Memoria Virtual
¿Qué es?	Un chip físico de hardware instalado en la placa madre.	Un mecanismo conceptual/lógico gestionado por el **kernel** del Sistema Operativo.
Ubicación	Módulos de memoria RAM (tarjetas físicas).	Utiliza una parte del almacenamiento secundario (Disco Rígido o SSD / Archivo de *Swap*).
Velocidad	**Extremadamente rápida.** La CPU accede a ella en nanosegundos.	**Mucho más lenta.** Al depender del disco (incluso en un SSD rápido), el acceso toma microsegundos o milisegundos.
Capacidad	Limitada físicamente por la cantidad de módulos instalados (ej. 8 GB, 16 GB, 32 GB).	Expandible. Su límite teórico depende del espacio libre disponible en el disco rígido.

Característica	Memoria RAM	Memoria Virtual
Costo	Alto por gigabyte.	Gratuito (utiliza el espacio libre del disco que ya compraste).
Acceso de la CPU	La CPU y la MMU acceden a ella **directamente**.	La CPU no puede ejecutar instrucciones directamente desde ahí; el kernel debe **mover los datos a la RAM primero**.

¿Cómo funcionan juntas?

Imagina un taller de carpintería:

1. **La Memoria RAM es el Banco de Trabajo:** Es el espacio físico donde el carpintero (la CPU) tiene las herramientas y piezas con las que está trabajando **en este preciso momento**. Todo lo que esté en la mesa se toma de inmediato.
2. **La Memoria Virtual es el Depósito o Almacén (Disco):** Si la mesa de trabajo (RAM) se llena de cajas y materiales, el carpintero no se detiene. Llama a un asistente (el **Kernel**) para que se lleve las cajas que no está usando en ese instante y las guarde temporalmente en el depósito (Disco).
3. **El intercambio (*Swapping*):** Cuando el carpintero vuelve a necesitar una pieza guardada en el depósito, el asistente va por ella, despeja un lugar en la mesa de trabajo moviendo otra cosa al depósito y le entrega la pieza requerida.

Resumen

La **RAM** es la memoria física real y rápida del equipo, mientras que la **Memoria Virtual** es la "ilusión" que crea el sistema operativo usando el disco rígido para hacerle creer a las aplicaciones que la computadora tiene más RAM de la que físicamente posee.

KERNEL

El kernel, también conocido como núcleo, es el componente fundamental de cualquier sistema operativo. Actúa como el puente o intermediario esencial entre el hardware de la computadora (el procesador, la memoria, el disco duro, la tarjeta de red, etc.) y las aplicaciones de software que utilizas.

Su función principal es gestionar eficientemente los recursos del hardware para que las aplicaciones puedan ejecutarse de manera coordinada, segura y eficiente. El kernel se carga en la memoria al iniciar el sistema y permanece allí hasta que se apaga la computadora.

Sus responsabilidades clave incluyen:

- **Gestión de Procesos:** Administra la ejecución de los programas (procesos). Decide qué proceso obtiene tiempo de CPU y por cuánto tiempo, crea, suspende y finaliza procesos, y maneja la comunicación entre ellos.
- **Gestión de Memoria:** Controla el acceso a la memoria RAM. Asigna porciones de memoria a los procesos para que puedan cargarse y ejecutarse, y asegura que un proceso no acceda a la memoria de otro sin autorización.
- **Gestión de Dispositivos (Entrada/Salida):** Facilita la comunicación entre el software y los periféricos (teclado, ratón, pantalla, discos duros, impresoras, etc.). Utiliza controladores de dispositivos (drivers) para traducir las solicitudes del software a comandos que el hardware puede entender.
- **Gestión del Sistema de Archivos:** Organiza y gestiona cómo se almacenan los datos en los dispositivos de almacenamiento (disco duro, SSD, USB), creando una estructura de archivos y directorios para que las aplicaciones puedan leer y escribir datos.
- **Seguridad y Control de Acceso:** Implementa mecanismos para proteger el sistema de accesos no autorizados y para asegurar que cada usuario y proceso solo tenga acceso a los recursos permitidos.

Modos de Operación (Modo Usuario vs. Modo Kernel):

- Para proteger la estabilidad y seguridad del sistema, el kernel opera en un modo privilegiado conocido como **Modo Kernel** (o Modo Supervisor). En este modo, el kernel tiene acceso total e irrestricto a todo el hardware del sistema y a todas las instrucciones del procesador.
- Por otro lado, las aplicaciones de usuario (como tu navegador web, editor de texto o juegos) se ejecutan en un modo restringido llamado **Modo Usuario**. Las aplicaciones en Modo Usuario no pueden acceder directamente al hardware ni a la memoria de otras aplicaciones o del kernel. Cuando una aplicación necesita realizar una operación privilegiada (como leer un archivo o enviar datos por la red), debe realizar una **Llamada al Sistema (System Call)**. Esta llamada actúa como una puerta de enlace segura que

solicita al kernel que realice la operación en su nombre. El kernel verifica la solicitud, la ejecuta de manera segura en Modo Kernel y luego devuelve el resultado a la aplicación en Modo Usuario.

¿Cuál es la diferencia entre Kernel e ISA (Instruction Set Architecture)?

- Esta es una distinción fundamental pero que a menudo causa confusión. Aunque ambos interactúan con el hardware, operan en niveles muy diferentes y tienen propósitos distintos.

Característica	Kernel (Núcleo del Sistema Operativo)	ISA (Instruction Set Architecture)
Definición	Software fundamental que gestiona el hardware y sirve como intermediario para las aplicaciones.	Es la especificación de diseño que define el conjunto de instrucciones que un procesador (CPU) puede ejecutar. Es la "interfaz" entre el software y el hardware.
Nivel	Capa de software de sistema. Se ejecuta sobre el hardware.	Es una especificación de hardware y software que define cómo debe comportarse el hardware.
Función Principal	Gestionar recursos (CPU, memoria, dispositivos), planificar procesos, proporcionar seguridad, etc.	Definir qué comandos entiende la CPU (como ADD, MOVE, LOAD, STORE, JUMP) y cómo se codifican.
Ejemplos	Kernel de Linux (usado en Ubuntu, Android), Kernel de Windows (NT), Kernel de macOS (XNU).	x86 (usado en la mayoría de PCs con Intel/AMD), ARM (usado en la mayoría de smartphones y tablets), RISC-V.
Relación	El kernel está escrito utilizando las instrucciones definidas por la ISA del procesador en el que se ejecutará.	La ISA define las reglas que el kernel debe seguir para controlar el hardware a través de instrucciones de bajo nivel.
Abstracción	Proporciona abstracciones de alto nivel para que las aplicaciones no tengan que preocuparse por los detalles específicos del hardware (por	Es una abstracción de bajo nivel que oculta los detalles microarquitectónicos de cómo el procesador implementa internamente las instrucciones

Característica	Kernel (Núcleo del Sistema Operativo)	ISA (Instruction Set Architecture)
	ejemplo, el kernel presenta "archivos" en lugar de sectores de disco).	(como el pipeline o las unidades de ejecución).

¿Cómo se relacionan?

Cuando una aplicación quiere leer un archivo:

1. La aplicación ejecuta una **system call**.
2. El control pasa al kernel.
3. El kernel usa drivers y mecanismos internos.
4. El kernel ejecuta instrucciones compatibles con la ISA del procesador.
5. El hardware realiza la operación.
6. El resultado vuelve a la aplicación.
7.

En resumen:

- **ISA** es el **"manual de instrucciones"** de un procesador específico. Define qué comandos puede entender.
- **El Kernel** es el **"gerente general"** de la computadora. Es un programa de software complejo que utiliza el manual de instrucciones (ISA) del procesador para controlar eficientemente todos los recursos del hardware y proporcionar un entorno seguro y organizado para que se ejecuten otras aplicaciones.

El kernel necesita una ISA para funcionar, pero la ISA es solo una definición del hardware, no es el software de control en sí.

Arquitecturas Monolítica, Microkernel e Híbrida.



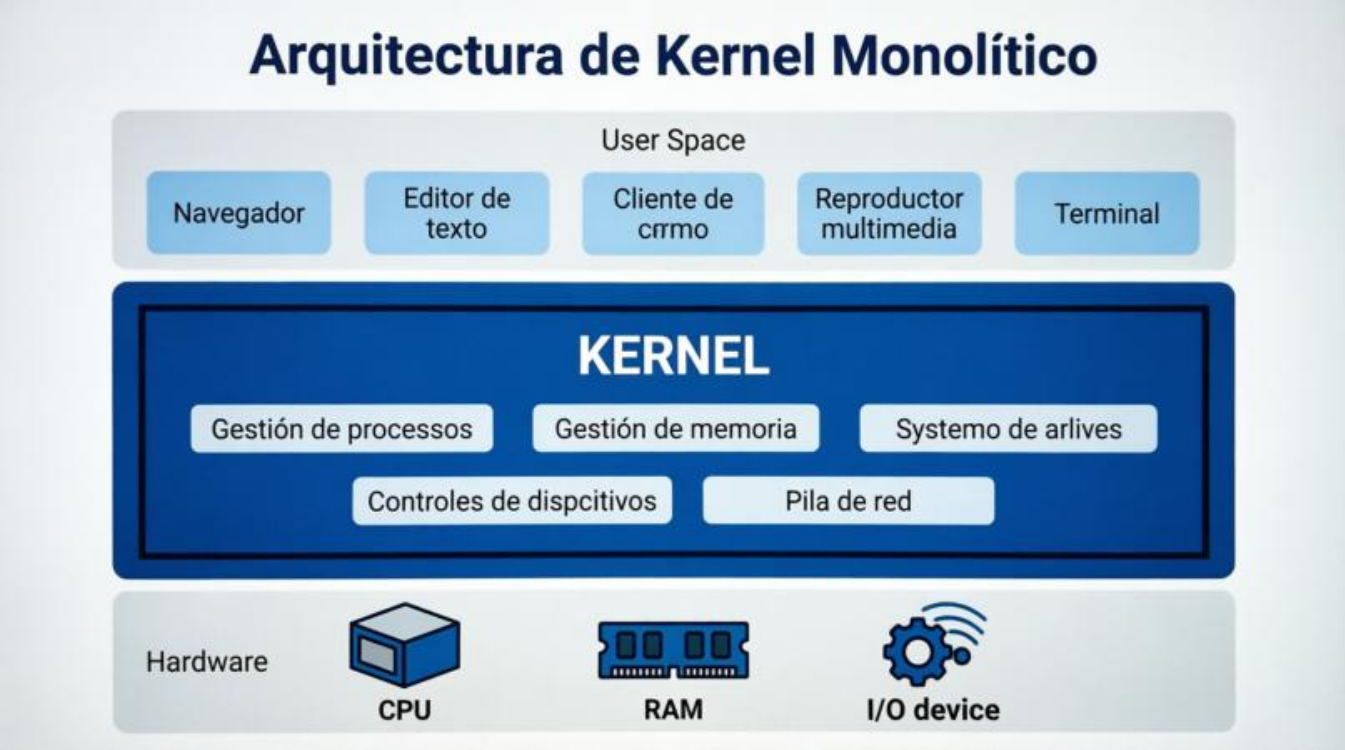
1. Kernel Monolítico (Monolithic Kernel):

En este diseño, todo el sistema operativo (gestión de procesos, gestión de memoria, sistemas de archivos, controladores de dispositivos, etc.) se ejecuta en un solo y gran bloque de código en el espacio del núcleo (Modo Kernel).

- **Ventajas:**
    - **Rendimiento:** Al estar todo en el mismo espacio de memoria, la comunicación entre componentes es muy rápida y eficiente (llamadas a funciones directas en lugar de paso de mensajes).
    - **Madurez:** Es una arquitectura muy probada y utilizada.
- **Desventajas:**
    - **Estabilidad:** Si un componente (como un controlador de dispositivo defectuoso) falla, puede provocar el fallo de todo el kernel y, por tanto, de todo el sistema.

- **Complejidad:** Al ser un bloque de código masivo, es más difícil de mantener, extender y depurar. Una modificación en una parte puede afectar inesperadamente a otras.
- **Seguridad:** Un atacante que logre comprometer un componente puede obtener acceso a todo el kernel.

Ejemplos: El kernel de Linux, los kernels tradicionales de Unix (como BSD), y las versiones iniciales de Windows.



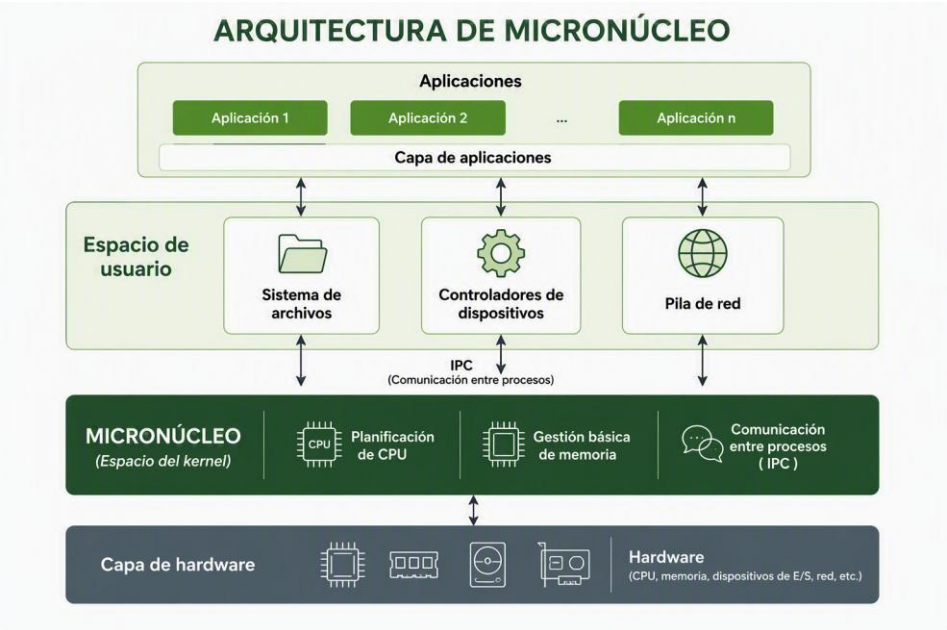
2. Microkernel (Micronúcleo):

Este diseño sigue la filosofía opuesta. El microkernel en sí es muy pequeño y solo contiene las funciones absolutamente esenciales que deben ejecutarse en Modo Kernel, como la gestión básica de la memoria, la planificación de procesos y los mecanismos de comunicación entre procesos (IPC - Inter-Process Communication).

La mayoría de los servicios tradicionales del kernel, como los sistemas de archivos, los controladores de dispositivos y las pilas de red, se ejecutan como procesos de usuario separados y aislados en el espacio de usuario (Modo Usuario), conocidos como **servidores**.

- **Ventajas:**
    - **Estabilidad y Fiabilidad:** Si un servidor (como el sistema de archivos) falla, solo ese servidor se ve afectado, y el microkernel principal sigue funcionando. El servidor fallido puede incluso reiniciarse automáticamente.
    - **Flexibilidad y Modularidad:** Es más fácil añadir o eliminar servicios del sistema sin afectar el núcleo. Los componentes están bien aislados.
    - **Seguridad:** El microkernel tiene una superficie de ataque mucho menor. Comprometer un servidor no compromete todo el sistema.
- **Desventajas:**
    - **Rendimiento:** La comunicación entre los servidores de usuario y el microkernel (y entre los propios servidores) requiere el paso de mensajes (IPC), lo que implica más overhead y cambios de contexto (context switches) que las llamadas a funciones directas en un kernel monolítico, lo que puede degradar el rendimiento.
    - **Complejidad de Diseño:** Implementar un sistema de paso de mensajes eficiente y gestionar múltiples servidores independientes es complejo.

Ejemplos: MINIX, QNX (usado en sistemas embebidos críticos), L4, GNU Hurd. El kernel de macOS (XNU) tiene una base microkernel (Mach) pero incluye muchos servicios monolíticos, por lo que a menudo se clasifica como híbrido.

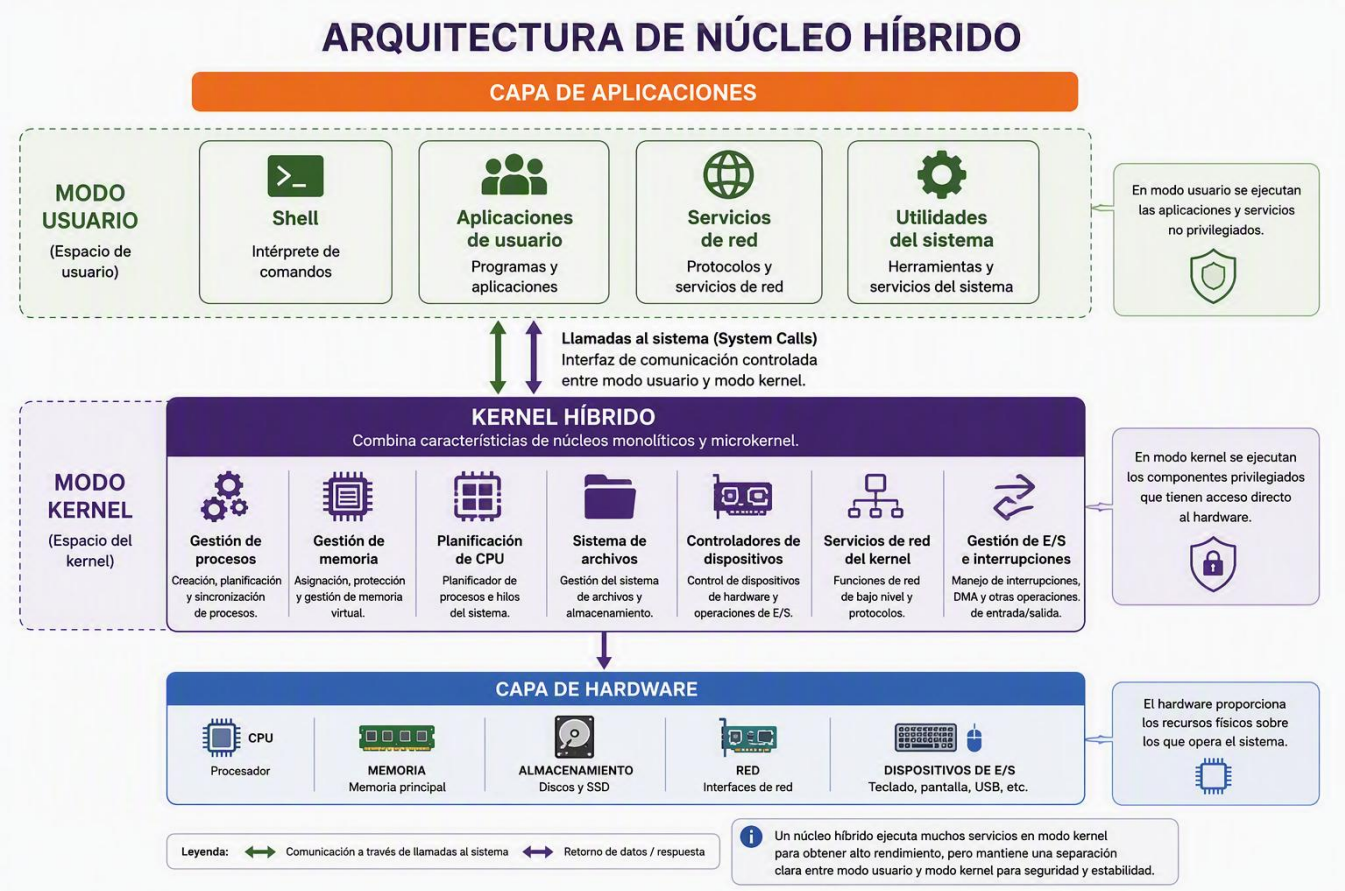


3. Kernel Híbrido (Hybrid Kernel):

Esta arquitectura es un enfoque pragmático que intenta combinar las ventajas de rendimiento de un kernel monolítico con el modularidad de un microkernel. Al igual que un microkernel, separa algunos servicios en Modo Usuario, pero permite que otros componentes críticos de rendimiento (como los controladores de gráficos o los sistemas de archivos más utilizados) se ejecuten en Modo Kernel para una comunicación más rápida.

- **Ventajas:** Ofrece un buen equilibrio entre rendimiento y modularidad. Muchos sistemas modernos optan por este enfoque para obtener lo mejor de ambos mundos.
- **Desventajas:** Puede volverse complejo de diseñar y mantener, ya que la distinción de qué componentes van en qué espacio puede volverse borrosa.

Ejemplos: El kernel de Windows NT (usado en Windows XP y versiones posteriores), macOS (con su kernel XNU).



Para entender la diferencia de forma muy sencilla, imagínate el kernel como si fuera **el equipo de administración de un hotel**:

1. Kernel Monolítico: *El hotel con un solo gran equipo todoterreno*

- **La idea:** Todo el personal (recepción, limpieza, seguridad, mantenimiento, cocina) trabaja dentro del mismo departamento principal y todos tienen acceso total a todo el edificio.
- **Cómo funciona:** Cuando necesitas algo, cualquier empleado lo resuelve directamente ahí mismo sin pedir permisos a otros departamentos.
- **Lo bueno:** Es súper rápido y eficiente porque todo está en el mismo lugar y la comunicación es directa.
- **Lo malo:** Si el encargado de la limpieza comete un error grave y provoca un cortocircuito, **se apaga todo el hotel** (si un controlador de video o red falla, se cae todo el sistema operativo).
- **Ejemplo:** Linux (Android, Ubuntu) y los sistemas Unix tradicionales.

2. Micronúcleo (Microkernel): *El hotel con departamentos independientes y contratistas*

- **La idea:** La administración central es minúscula. Solo se encarga de lo estrictamente vital (la llave maestra y la contestadora de llamadas). Todo lo demás (limpieza, cocina, seguridad) son empresas externas que trabajan en sus propias oficinas.
- **Cómo funciona:** Si pides comida, la recepción no cocina; le manda un mensaje al restaurante externo, este la prepara y te la envía.
- **Lo bueno:** Es extremadamente seguro y estable. Si el servicio de restaurante quiebra o falla, **el resto del hotel sigue funcionando perfectamente** y solo reinicias el servicio de comida.
- **Lo malo:** Es un poco más lento, porque enviar mensajes de ida y vuelta entre la administración central y las empresas externas toma tiempo extra.
- **Ejemplo:** QNX (usado en autos y sistemas médicos donde la falla no es una opción) o MINIX.

3. Kernel Híbrido: *El enfoque práctico (un punto medio)*

- **La idea:** Una mezcla de los dos anteriores. La administración central tiene una base pequeña como el micronúcleo, pero decide mantener a los servicios más usados y rápidos (como la limpieza de emergencia o el mantenimiento principal) dentro de la oficina central para no perder tiempo.
- **Cómo funciona:** Intenta mantener las cosas importantes separadas para que no falle todo el sistema, pero junta los servicios críticos de rendimiento en un solo lugar para que el sistema sea veloz.

- **Lo bueno:** Consigue un gran equilibrio entre la velocidad del monolítico y la organización/seguridad del micronúcleo.
- **Ejemplo:** Windows NT (Windows 10, 11) y macOS.

Resumen ultrarrápido:

- Monolítico: Todo en un solo paquete → Muy rápido, pero un fallo tira todo.
- Micronúcleo: Todo dividido en módulos independientes → Súper seguro y estable, pero un poco más lento.
- Híbrido: Mezcla de ambos → Busca el equilibrio ideal entre velocidad y seguridad.

Tipo	Analogía	Velocidad	Estabilidad
Monolítico	Todos en la misma habitación.	Muy rápida	Baja (si uno falla, caen todos)
Micronúcleo	Oficinas separadas, todo por mensajes.	Más lenta	Muy alta (los fallos se aíslan)
Híbrido	Cocina abierta + oficinas separadas.	Equilibrada	Equilibrada

En la vida real:

- Linux es predominantemente monolítico (por eso es tan rápido en servidores).
- Windows y macOS son híbridos (buscan que tu PC no se bloquee fácilmente, pero que los juegos y programas corran rápido).
- QNX o MINIX son micronúcleos (usados en aviones, coches o equipos médicos, donde un fallo no puede ser catastrófico).

1. ¿Qué kernels usan los servidores?

En el mundo de los servidores, la **eficiencia y el rendimiento** son lo más importante. Por eso:

- **El rey absoluto: Kernel Monolítico (Linux).** Más del 90% de los servidores del mundo (nube, páginas web, supercomputadoras) usan Linux (Ubuntu Server, Debian, RHEL, etc.). ¿Por qué? Porque su kernel monolítico maneja el hardware de forma directa y rapidísima, sin la lentitud de pasar mensajes entre capas.
- **El segundo lugar: Kernel Híbrido (Windows Server).** Muchas empresas lo usan por compatibilidad con sus sistemas internos. El kernel de Windows NT es híbrido: busca un

equilibrio para que sea rápido, pero que si un driver de impresora o red falla, no tumbe todo el servidor.

- **Casos muy específicos: Micronúcleo.** Servidores de telecomunicaciones, sistemas bancarios de alta seguridad o control industrial a veces usan QNX o MINIX, donde es más importante que el sistema *nunca* se caiga, aunque sea un poco más lento.

2. ¿Qué tiene que ver esto con los Hipervisores y Máquinas Virtuales?

Un **hipervisor** es, en esencia, un "sistema operativo de sistemas operativos". Su trabajo es crear y gestionar Máquinas Virtuales (VMs). Y aquí es donde el tipo de kernel es crucial.

Existen dos tipos de hipervisores, y su relación con el kernel es muy clara:

Hipervisor de Tipo 1 (Bare-Metal o "sobre el metal" - Nativo)

- **Cómo funciona:** Se instala **directamente sobre el hardware físico**, sin un sistema operativo tradicional debajo.
- **Relación con el kernel:** El propio hipervisor **actúa como un kernel altamente especializado** (muy parecido a un micronúcleo o un monolítico minimalista). Su único trabajo es gestionar el hardware real y repartirlo entre las máquinas virtuales.
- **Ejemplos:** VMware ESXi, Proxmox VE, Microsoft Hyper-V (la capa base), Xen.
- **Analogía:** Es como comprar un edificio vacío y poner un "Gerente Supremo" (el hipervisor) que divide el edificio en varios locales independientes.

Hipervisor de Tipo 2 (Hosted o "alojado")

- **Cómo funciona:** Se instala como un **programa más** dentro de un sistema operativo normal (tu Windows, macOS o Linux de escritorio).
- **Relación con el kernel:** Depende totalmente del kernel del sistema operativo anfitrión (monolítico o híbrido) para acceder al hardware. Es más lento porque las peticiones deben pasar por el kernel del SO anfitrión, luego por el hipervisor, y luego a la VM.
- **Ejemplos:** VirtualBox, VMware Workstation, Parallels Desktop.
- **Analogía:** Es como alquilar un local dentro de un centro comercial. El gerente del centro comercial (kernel de tu Windows) tiene la última palabra sobre el agua y la luz, y tú (el hipervisor) solo gestionas tu pequeño local.

3. El concepto clave: "El Kernel Invitado" (Guest Kernel)

Aquí está la magia: **Cada Máquina Virtual cree que es una computadora física independiente.**

Por lo tanto, **cada VM instala y ejecuta SU PROPIO kernel**:

1. El hardware real es gestionado por el **kernel del hipervisor** (Tipo 1).
2. El hipervisor crea hardware "falso" (virtualizado) para la VM.
3. Tú instalas, por ejemplo, Linux en esa VM. Ese Linux carga **su propio kernel monolítico**, el cual cree que está hablando con un procesador y una memoria reales, pero en realidad le está hablando al hipervisor.

Para entender la relación entre el Kernel, el Hipervisor y las Máquinas Virtuales (VMs), imagina un edificio de departamentos:

- Servidor Físico (Bare Metal): Es el terreno y la estructura del edificio.
- Hipervisor (Tipo 1): Es el administrador del edificio. Su trabajo es dividir el hardware real (CPU, RAM, Disco) en "departamentos" independientes.
- Máquina Virtual (VM): Es cada departamento individual. Cada VM cree que es una computadora completa y privada.
- Kernel del Sistema Operativo de la VM: Es la instalación interna de cada departamento. Cada máquina virtual tiene su propio kernel independiente corriendo dentro de ella.

Resumen visual del flujo:

➔Aplicación en la VM

➔ Kernel de la VM (Invitado)

➔ Hypervisor

➔ Kernel del Hypervisor

➔ Hardware Real.

Máquinas Virtuales

Una **máquina virtual (VM)** es un ordenador "simulado" que funciona dentro de otro ordenador físico.

Imagina que tu computadora es un edificio. Dentro de ese edificio puedes crear varios apartamentos independientes. Cada apartamento tiene su propio sistema operativo, programas y configuración. Esos apartamentos serían las máquinas virtuales.



¿Cómo funciona?

1. **Hardware físico**
    - Procesador (CPU)
    - Memoria RAM
    - Disco SSD/HDD
    - Tarjeta de red
2. **Hipervisor**
    - Es el software que administra las máquinas virtuales.
    - Ejemplos: VMware, VirtualBox, KVM, Hyper-V.
    - Reparte los recursos físicos entre las VMs.
3. **Máquinas virtuales**
    - Cada una tiene su propio sistema operativo.

- Actúan como si fueran computadoras independientes.

¿Para qué se usan?

1. Ejecutar varios sistemas operativos en una sola máquina

Por ejemplo, en un ordenador con Windows puedes ejecutar:

- Ubuntu
- Debian
- Windows Server

al mismo tiempo.

2. Pruebas y desarrollo

Muy útil para desarrolladores e investigadores.

Ejemplo:

- Pruebas una aplicación en Ubuntu.
- Si algo sale mal, eliminas la VM y creas otra.

3. Servidores

En centros de datos es habitual que un servidor físico aloje muchas VMs:

Servidor físico

├─ VM 1 → Base de datos

├─ VM 2 → Servidor web

├─ VM 3 → Correo

└─ VM 4 → Aplicación empresarial

Ventajas

✅ Aislamiento entre sistemas
✅ Mejor aprovechamiento del hardware
✅ Facilidad para hacer copias y snapshots
✅ Permite ejecutar varios sistemas operativos simultáneamente

Desventajas

❌ Consumen RAM y CPU adicionales
❌ Menor rendimiento que ejecutar directamente sobre el hardware
❌ Requieren una buena gestión de recursos

Ejemplo real

Supón que tienes:

- CPU: 8 núcleos
- RAM: 32 GB
- SSD: 1 TB

Podrías crear:

VM	Sistema Operativo	RAM
VM1	Ubuntu Server	8 GB
VM2	Windows Server	8 GB
VM3	Debian	4 GB
VM4	Kali Linux	4 GB

Todas funcionando simultáneamente sobre el mismo equipo físico.



Máquinas Virtuales vs Docker

La diferencia más importante es esta:

Una máquina virtual virtualiza un ordenador completo. Docker virtualiza aplicaciones y procesos.

Analogía sencilla. Imagina un edificio:

Máquina Virtual (VM)

Cada inquilino tiene un apartamento completo:

- Cocina
- Baño
- Dormitorio
- Electricidad propia

Cada apartamento funciona de forma independiente.

1 Hardware
└── Hipervisor
├── VM 1 (Ubuntu + Kernel Linux)
├── VM 2 (Windows + Kernel Windows)
└── VM 3 (Debian + Kernel Linux)

Cada VM incluye:

- Sistema operativo completo
- Kernel propio
- Librerías
- Aplicaciones

Docker (Contenedores)

Ahora imagina un edificio compartido donde todos usan la misma infraestructura:

Hardware

└── Sistema Operativo Host (Linux)

└── Docker Engine

├── Contenedor A (Aplicación Web)

├── Contenedor B (Base de Datos)

└── Contenedor C (API)

Los contenedores:

- Comparten el kernel del host.
- Solo llevan la aplicación y sus dependencias.
- No necesitan un sistema operativo completo.

Comparación rápida

Característica	VM	Docker
Sistema operativo propio	✅ Sí	❌ No
Kernel propio	✅ Sí	❌ Comparte el del host
Consumo de RAM	Alto	Bajo
Tiempo de arranque	Segundos o minutos	Milisegundos o segundos
Aislamiento	Muy alto	Alto
Portabilidad	Buena	Excelente
Tamaño típico	GB	MB

Ejemplo práctico

Supongamos que desarrollas una aplicación en Python.

Con VM

VM Ubuntu

├── Ubuntu completo

├── Python

├── Librerías

└── Aplicación

Puede ocupar varios GB de disco.

Con Docker

Contenedor

├── Python

├── Librerías

└── Aplicación

Puede ocupar unos cientos de MB.

¿Cuándo usar una VM?

✅ Necesitas varios sistemas operativos distintos

Ejemplo:

- Windows Server
- Ubuntu
- CentOS

en el mismo hardware.

✅ Necesitas un aislamiento muy fuerte.

✅ Quieres probar sistemas operativos completos.

¿Cuándo usar Docker?

✅ Microservicios

✅ APIs

✅ Aplicaciones web

✅ Ciencia de datos

✅ Entornos reproducibles

✅ Despliegues en la nube

Por ejemplo, para tu trabajo con **Python, Qiskit y QSVM**, Docker es muy útil porque puedes encapsular exactamente:

```
Python 3.11
Qiskit 1.x
scikit-learn
numpy
pandas
```

y cualquier persona puede ejecutar el mismo entorno sin sufrir problemas de versiones o dependencias.

Docker dentro de una VM

En la práctica empresarial es muy común:

```
Servidor físico
└── VM Ubuntu
└── Docker
├── API
├── PostgreSQL
├── Redis
└── Nginx
```

Es decir:

- La VM proporciona aislamiento a nivel de sistema operativo.
- Docker proporciona aislamiento a nivel de aplicación.

Regla fácil para recordar

VM = virtualiza computadores completos.

Docker = virtualiza aplicaciones.

Por eso Docker suele ser más ligero, más rápido y más fácil de desplegar, mientras que una VM ofrece un aislamiento más fuerte y permite ejecutar sistemas operativos diferentes en el mismo hardware.

Una arquitectura moderna muy común es combinar **VMs + Docker**, aprovechando las ventajas de ambos.

Arquitectura simplificada

¿Qué hace cada capa?

1. Hardware físico

Es el servidor real:

- CPU
- Memoria RAM
- SSD
- Red

Por ejemplo:

- 32 CPU
- 128 GB RAM
- 4 TB SSD

2. Hipervisor

Administra las máquinas virtuales.

Ejemplos:

- VMware ESXi
- Hyper-V

Su función es repartir recursos:

Servidor

├— VM Linux (16 GB)

├── VM Windows (32 GB)

└── VM Backup (16 GB)

3. Máquina Virtual Linux

Dentro de esta VM instalamos Docker.

VM Ubuntu

├── Docker

├── Monitorización

└── Seguridad

La VM proporciona:

- Sistema operativo completo
- Kernel Linux
- Aislamiento frente a otras VMs

¿Por qué no usar solo Docker?

Porque la VM aporta:

✅ Mayor aislamiento
✅ Seguridad adicional
✅ Control de recursos
✅ Separación entre entornos

Si un contenedor falla, no afecta a otras VMs.

¿Por qué no usar solo VMs?

Porque Docker ofrece:

✅ Arranque en segundos
✅ Menor consumo de RAM
✅ Despliegue sencillo
✅ Reproducibilidad

Crear 20 contenedores es mucho más ligero que crear 20 máquinas virtuales.

Regla de oro

Hardware
↓
Hipervisor
↓
VM Linux
↓
Docker
↓
Aplicaciones

La VM aísla sistemas operativos. Docker aísla aplicaciones.

Por eso, en entornos empresariales actuales, la arquitectura "VM + Docker" suele considerarse una de las combinaciones más prácticas y equilibradas.

La diferencia con los "Contenedores" (Docker / Kubernetes)

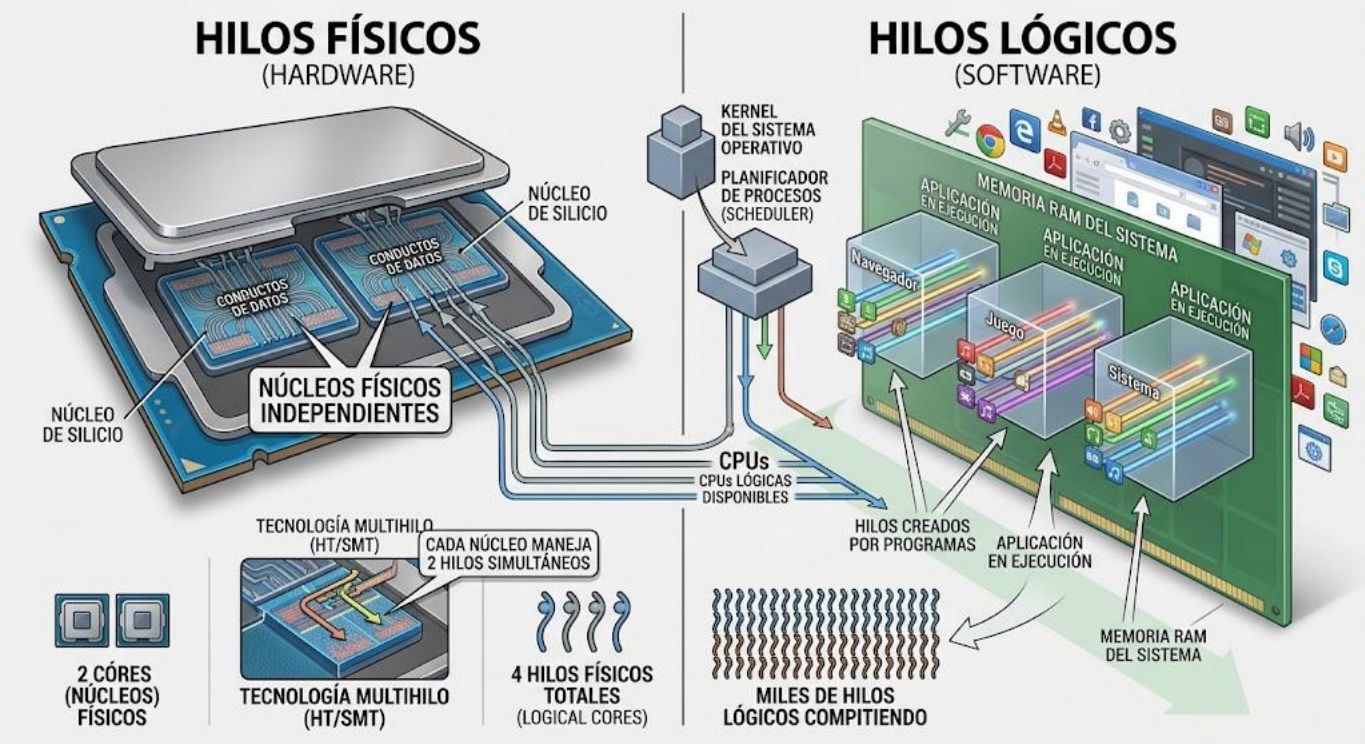
Esto te ayudará a entender por qué el kernel es tan importante hoy en día:

- **Máquina Virtual (VM):** Tiene **su propio kernel** completo. Es pesada, tarda más en arrancar, pero está totalmente aislada. (Como un restaurante independiente con su propia cocina y gerente).
- **Contenedor (Docker): NO tiene su propio kernel**. Todos los contenedores en un servidor **comparten el mismo kernel monolítico del servidor anfitrión** (Linux). Solo empaquetan la aplicación y sus librerías. Por eso son ultraligeros y arrancan en milisegundos. (Como varios puestos de comida que comparten la misma cocina y el mismo gerente del centro comercial).

En conclusión:

Sí, los kernels son la base de todo. Los servidores usan mayoritariamente kernels **monolíticos** (Linux) por velocidad. Cuando añades virtualización, el **hipervisor** actúa como un "meta-kernel" que engaña a los kernels de las máquinas virtuales para que puedan convivir en el mismo hardware sin pisarse entre ellos.

Thread – Multithreading



HILOS FÍSICOS (HARDWARE) - A la izquierda: Aquí puedes ver la anatomía de un procesador de 4 núcleos físicos + 8 hilos. He puesto una lupa sobre un núcleo para mostrar cómo la tecnología de multihilo (Hyper-Threading/SMT) crea dos vías lógicas dentro de un solo motor físico. El Sistema Operativo ve esto como 8 CPUs lógicas disponibles, aunque solo haya 4 núcleos reales.

HILOS LÓGICOS (SOFTWARE) - A la derecha: Esta parte muestra qué sucede en la memoria RAM y dentro de tus aplicaciones. Cada cubo representa una aplicación (como tu Navegador, Spotify o un Videojuego). Dentro de cada aplicación, puedes ver **múltiples hilos de software** individuales que trabajan en paralelo. Hay miles de ellos ejecutándose en la RAM, y todos compiten por un "turno" de ejecución en los 8 hilos de hardware que tiene tu CPU.

Hyper-Threading y **SMT** son exactamente la misma tecnología, pero con nombres diferentes según el fabricante de procesadores:

- **Hyper-Threading (HT):** Es el nombre comercial que usa **Intel**.
- **SMT (*Simultaneous Multithreading* / Multihilo Simultáneo):** Es el término técnico general de la industria y el nombre comercial que utiliza **AMD**.

¿En qué consiste esta tecnología?

Es una técnica a nivel de hardware que le permite a un solo núcleo físico de la CPU actuar como si fuera dos núcleos lógicos ante el sistema operativo.

Para entenderlo a nivel de circuitos:

1. **Un núcleo tradicional (sin SMT/HT):** Tiene una sola "tubería" (*pipeline*) de entrada para recibir instrucciones. Si el núcleo está procesando un cálculo y tiene que esperar a que lleguen datos desde la memoria RAM (lo cual toma tiempo en escala de microsegundos), el núcleo se queda desocupado (*idle*) sin hacer nada.
2. **Un núcleo con SMT/HT:** Duplica ciertas partes ligeras del núcleo físico (como los registros de estado, los punteros de instrucciones y la tabla de arquitectura), pero no duplica las unidades de cálculo pesado (como la ALU).

Al tener dos vías de entrada guardadas en el propio núcleo, si la Vía 1 se frena esperando datos de la RAM, la Vía 2 aprovecha de inmediato los circuitos de cálculo libres para procesar otra instrucción.

¿Duplica el rendimiento?

No. Como los dos hilos comparten las mismas herramientas físicas de cálculo del núcleo, no obtienes el doble de velocidad (eso solo se logra añadiendo otro núcleo físico real).

En la práctica, SMT/Hyper-Threading ofrece un incremento de rendimiento de entre un 20% y un 30% en tareas pesadas que pueden paralelizarse (renderizado 3D, edición de video, compilación de código o ejecución de múltiples máquinas virtuales).

Resumen

Hyper-Threading (Intel) y SMT (AMD) son el truco de arquitectura que permite que un procesador de, por ejemplo, 6 núcleos físicos puedan reportarle al sistema operativo que tiene 12 hilos/núcleos lógicos disponibles para trabajar.

Resumen gráfico

Concepto	**Hilos de Software**	**Hilos de Hardware**
Creados por	Los programadores y las aplicaciones.	Los ingenieros de chips (Intel / AMD).
Cantidad	Ilimitada (pueden ser miles en ejecución).	Fija por el procesador (ej. 8, 12, 16).

Concepto	Hilos de Software	Hilos de Hardware
Ubicación	Memoria RAM (administrados por el Kernel).	Registros físicos en la CPU.
Función	Dividir un programa en multitareas.	Maximizar el uso de los núcleos físicos.

